

Generalizable and Per-Scene NeRF Architectures under Sparse-View Conditions: A Systematic Benchmarking Methodology

Abstract—This work proposes a methodology for systematically evaluating the trade-offs between computational performance and reconstruction fidelity of neural radiance field (NeRF) architectures under sparse-view conditions. Four representative architectures are evaluated using visual fidelity metrics, including Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS), together with computational performance metrics, including training convergence time, Video Random Access Memory (VRAM) footprint, and inference latency. The experimental results reveal a clear trade-off between reconstruction fidelity and computational efficiency. Per-Scene architectures consistently outperform Generalizable models in visual quality, achieving up to 27.11 dB PSNR, 0.875 SSIM, and 0.171 LPIPS, while also providing significantly higher inference throughput and lower memory consumption. In contrast, generalizable models offer greater flexibility for acquisition-constrained scenarios but exhibit limited reconstruction fidelity and unstable optimization under severe sparse-view conditions. The proposed methodology and the resulting quantitative analyses establish a standardized and reproducible framework for benchmarking heterogeneous neural rendering architectures, enabling systematic model selection according to computational and acquisition constraints.

I. INTRODUCTION

Neural Radiance Fields (NeRFs) have consolidated themselves in recent years as the state-of-the-art for novel view synthesis, in both industry and academia [1]–[3]. However, the most advanced neural rendering techniques require considerable computational resources and multi-view datasets for training. Both challenges make practical applications, such as mobile scanning, virtual try-ons, and synthesis in reduced datasets, unfeasible. To address this problem, several methods have been proposed to advance the state of the art, including Instant-NGP (*Instant Neural Graphics Primitives*), Nerfacto, PixelNeRF, and GNT (*Generalizable NeRF Transformer*) [2], [4]–[6]. These methods investigate different aspects of neural rendering, such as spatial *encoding* [4], hybrid architectures that combine Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) [6]–[8], and model pre-computation (*baking*) into compact representations with a reduced memory footprint [9].

In this context, this work investigates the application of different NeRF models and their respective optimization techniques to sparse-view datasets by developing a methodology for evaluating the trade-off between computational performance and the visual quality of the reconstructed volumes. Reconstruction fidelity was assessed using three widely adopted image quality metrics: PSNR (*Peak Signal-to-Noise Ratio*),

SSIM (*Structural Similarity Index Measure*) [10], and LPIPS (*Learned Perceptual Image Patch Similarity*) [11]. The operational cost of each model was quantified using system-level performance metrics, including training convergence time, inference speed, and memory footprint, particularly VRAM (*Video Random Access Memory*) consumption. Although previous studies have investigated NeRFs under limited-data settings [12], [13], and benchmarking frameworks such as NerfBaselines [14] have evaluated traditional and per-scene NeRF models, a systematic comparison of generalizable neural rendering architectures employing convolutional and attention-based feature extractors under controlled hardware conditions and reproducible data splits remains absent from the literature. To bridge this gap, this work develops a systematic evaluation methodology for quantitatively assessing the trade-offs between computational efficiency and reconstruction fidelity across four neural rendering architectures under three sparse-view data regimes. Thus, the main contributions of this work are: (i) the formulation of a unified evaluation protocol for few-shot and per-scene NeRF architectures, ensuring reproducibility through standardized data partitions, controlled execution environments, and consistent evaluation metrics across three sparse-view data regimes; and (ii) the implementation and public release of integration wrappers that extend the Nerfstudio framework with PixelNeRF and GNT, providing a common experimental infrastructure for standardized benchmarking of heterogeneous neural rendering architectures.

II. THEORETICAL BACKGROUND

Sparse View Datasets. In this work, this term refers to image datasets lacking redundancy or camera pose variability for a given object. Fields like histology, holotomography, and fluorescence confocal microscopy exemplify this challenge, where multi-view acquisition is physically limited, biologically invasive, and inherently sparse [15].

Neural Radiance Fields (NeRF). NeRFs utilize Multilayer Perceptrons (MLPs) to approximate a continuous 5D function [16]. This function takes a spatial position $\mathbf{x} = (x, y, z)$ and a viewing direction $\mathbf{d} = (\theta, \phi)$ as inputs, outputting the volumetric density $\sigma(\mathbf{x})$ and associated color $\mathbf{c} = (r, g, b)$:

$$\gamma(\mathbf{x}, \mathbf{d}) \rightarrow \mathbf{v}, \quad F'_\theta(\mathbf{v}) \rightarrow (\sigma, \mathbf{c}), \quad \sigma \in \mathbb{R}_{\geq 0}, \quad \mathbf{c} \in [0, 1]^3$$

where θ represents the neural network weights, F'_θ is the original MLP, and γ is a positional encoder mapping inputs to a higher-dimensional feature vector $\mathbf{v}$.

Scene Representations. Subsequent works introduced hybrid representations that decouple the spatial structure from the neural network by replacing Fourier positional encoding with hash encoding. Instant-NGP [4] proposes a multiscale hash table encoding that efficiently maps spatial coordinates to feature vectors, drastically reducing per-scene optimization time. Nerfacto [2] extends this by combining proven heuristics from the literature, focusing on balancing visual quality and real-time rendering performance.

Volumetric Rendering. For a given neural radiance field F_θ , the expected color of a pixel is computed via volumetric rendering. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ with depth bounds (t_n, t_f) , the resulting color is given by the integral:

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt$$

where $T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$ denotes the accumulated transmittance, representing the probability that the ray does not hit any obstructive particle up to depth t . In practice, this integral is numerically approximated via alpha compositing over discrete samples along each ray during training.

Feature Extractor. The feature extractor encodes input images into representations that condition the neural network. PixelNeRF [5] uses a pre-trained ResNet to extract per-image feature maps; given a 3D point, its 2D projection onto the input views retrieves the corresponding spatial features, which are aggregated and passed to the radiance MLP. GNT (Generalizable NeRF Transformer) [6] replaces: (i) the feature extractor with a vision transformer to capture more global object features; and (ii) the radiance MLP with a cross-attention transformer that samples points along the ray and aggregates multi-view information, eliminating the explicit volume function dependency.

Few-Shot versus Per-Scene NeRFs. To reduce the need for dense multi-view training data, few-shot NeRFs generalize and synthesize novel views from sparse inputs by applying diverse concepts to the aforementioned NeRF components. These contrast sharply with per-scene methods. While per-scene models (e.g., Instant-NGP) optimize isolated data structures and suffer severe geometric degradation under view scarcity, generalizable few-shot architectures incorporate structural priors learned from large datasets. To maximize visual fidelity in generalizable models, Test-Time Adaptation (TTA) is frequently employed. Instead of purely zero-shot inference, TTA fine-tunes the network weights using the sparse views of the novel scene. This rapid optimization reduces the domain gap, though it introduces the risk of representational collapse (catastrophic forgetting) if the network overfits to the sparse inputs.

III. MATERIALS AND METHODS

This section presents the proposed methodology for systematically evaluating the trade-offs between computational

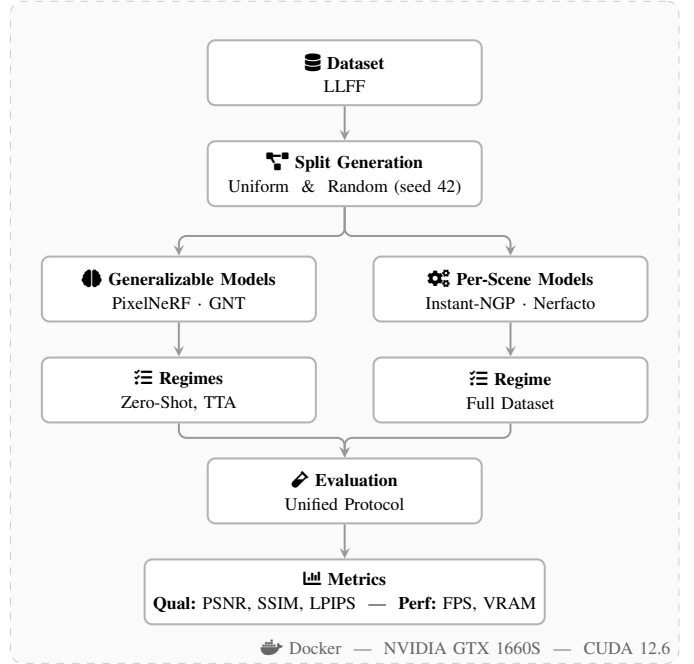


Fig. 1: Proposed methodology pipeline. The flowchart details the progression from data preparation to the bifurcated training paradigms, culminating in a unified evaluation protocol. The entire pipeline executes within an isolated containerized environment.

performance and reconstruction fidelity of NeRF models under sparse-view conditions. Figure 1 illustrates the proposed evaluation pipeline, which consists of three main stages: (i) dataset selection and sparse-view sampling; (ii) configuration and training of the selected neural rendering architectures according to their respective learning paradigms; and (iii) quantitative evaluation using standardized metrics to jointly characterize computational efficiency and reconstruction fidelity.

A. Dataset & Splits

The experiments were conducted on the LLFF (*Local Light Field Fusion*) dataset, which comprises 8 real-world scenes (*fern, flower, fortress, horns, leaves, orchids, room, trex*) captured with a forward-facing moving camera. Camera poses and depth bounds were obtained through COLMAP, following standard dataset procedures [17]. Furthermore, images were evaluated using specifically downscale of 8 resolution (504×378).

The complete data splitting protocol follows the few-shot evaluation configuration established by Niemeyer et al. [12]. Specifically, the testing split was generated by reserving images with indices that are multiples of 8 (`llffhold=8`), while the training views for the sparse regimes were strictly limited to $N \in \{3, 6, 10\}$. The selection of these training views was performed using two distinct strategies: (i) *uniform*, where views are sampled via linear interpolation over the dataset indices, distributing them evenly along the camera trajectory (a strategy that does not generalize to other datasets, since

LLFF indices intrinsically encode camera trajectory order); and (ii) *random*, where views are randomly selected using a fixed seed (42) to ensure reproducibility. Comparing these two strategies allows us to evaluate the models’ sensitivity to the spatial distribution of the input views, as demonstrated in Table I. Given the empirical advantage of uniform sampling (+0.54 dB PSNR on the *fern* scene), this strategy was adopted for all subsequent experiments.

TABLE I: Impact of the training view selection strategy (*uniform* versus *random*) on average visual quality. Evaluated under the TTA regime with $N = 10$ views on the *fern* scene.

Model	Split	PSNR (dB) $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Δ PSNR
PixelNeRF	Uniform	6.98	0.047	0.846	+0.54
	Random	6.44	0.042	0.840	
GNT	Uniform	10.34	0.179	0.738	+0.15
	Random	10.19	0.184	0.742	
Instant-NGP	Uniform	13.57	0.231	0.627	+0.06
	Random	13.51	0.229	0.630	
Nerfacto	Uniform	12.04	0.227	0.809	+0.11
	Random	11.93	0.226	0.865	

Beyond the core optimization hyperparameters, specific pipeline configurations were strictly enforced to guarantee stability and computational efficiency across the few-shot regimes. Notably, camera pose refinement was disabled for Nerfacto to prevent the catastrophic geometric collapse frequently observed in sparse-view scenarios. To minimize memory footprint and accelerate the Test-Time Adaptation phase, mixed-precision training was enabled across all architectures. For the hash-based models (Instant-NGP and Nerfacto), the multiscale hash table capacity was constrained to a maximum size of 2^{16} , with backgrounds explicitly set to white. Furthermore, structural hyperparameters for GNT were locked to 48 coarse and importance samples per ray, a transformer depth of 8, and a network width of 64, ensuring reproducible structural bounds during volume rendering and following the default configuration from the original paper [6].

B. Models & Training Regimes

The proposed strategy evaluates representative models for novel view synthesis under few-shot constraints, encompassing distinct architectural paradigms. The generalizable approaches selected were: (i) PixelNeRF [5], which utilizes Convolutional Neural Networks (CNNs) to extract spatial features and inject them into the renderer via projection; and (ii) Generalizable NeRF Transformer (GNT) [6], which replaces both the feature extractor and the differentiable renderer with cross-attention modules. As comparative baselines, two highly efficient per-scene architectures were chosen: Instant-NGP [4], which replaces traditional MLPs with a multiscale hash table encoding, and Nerfacto [2], an aggregate model combining state-of-the-art heuristics for rapid convergence and high fidelity.

The experimental execution was structured across three distinct regimes: (i) Zero-Shot: The ideal paradigm for extreme compute constraints, performing instant inference using

pre-trained weights without any local optimization on the novel scene; (ii) Test-Time Adaptation (TTA): An intermediate regime where the pre-trained generalizable models are fine-tuned using the sparse input views ($N \in \{3, 6, 10\}$), as reported in Table II); (iii) Per-Scene (100%): Representing the upper bound for visual fidelity, this regime optimizes the baseline models from scratch using the entirety of available images for a specific scene. It is crucial to note that during the Test-Time Adaptation (TTA) phase for the generalizable models, the core feature extraction networks (CNN and ViT) were kept frozen, updating only the terminal rendering layers. Preliminary ablation tests revealed that unfreezing the entire network under extreme data sparsity ($N \leq 10$) led to catastrophic forgetting and severe overfitting to the training poses, actively degrading novel view synthesis performance.

TABLE II: Impact of N training views on PSNR (dB) on the *room* scene.

Model	N=3	N=6	N=10
GNT (ZS)	12.96	12.85	12.85
GNT (TTA)	11.78	12.85	11.58
PixelNeRF (ZS)	7.35	7.15	7.29
PixelNeRF (TTA)	7.24	7.26	7.13

TABLE III: PSNR (dB) under varying learning rates (η). Ablation conducted on the *fern* scene under the TTA regime with $N = 10$ views.

Model (Regime)	η	PSNR
PixelNeRF (TTA)	10^{-4}	11.19
PixelNeRF (TTA)	5×10^{-4}	12.23
PixelNeRF (TTA)	10^{-3}	9.42
PixelNeRF (ZS)	N/A	6.64
GNT (TTA)	10^{-4}	12.96
GNT (TTA)	5×10^{-4}	12.96
GNT (TTA)	10^{-3}	12.85
GNT (ZS)	N/A	10.73

To ensure a strictly equitable evaluation protocol, all models were unified under the *Nerfstudio* framework [2]. Instant-NGP and Nerfacto are natively supported by the framework. Conversely, custom integration wrappers for PixelNeRF and GNT were developed and publicly released, manually porting their default components into semantic DataManager, Pipeline, and Model components to comply with Nerfstudio’s modular interface without altering their native inference logic. This unified environment leverages *tiny-cuda-nn* [18] for GPU acceleration and standardizes the logging infrastructure.

For consistency across the evaluation, all models were optimized using Adam with a standard learning rate of $\eta = 10^{-3}$, zero weight decay, 512 rays per batch in training and 128 in the evaluation step. Optimization budgets were scaled according to the regime: TTA configurations ran for 2500 iterations, while per-scene optimizations ran for 30000 iterations to ensure full convergence. Although per-architecture tuning of η would yield individually higher peak metrics (as evidenced by PixelNeRF’s 2.81 dB improvement under $\eta = 5 \times 10^{-4}$, Table III), such tuning introduces a confounding variable that conflates hyperparameter sensitivity with architectural capability. To isolate architecture as the primary experimental variable, a uniform $\eta = 10^{-3}$ was enforced across all models. This represents a deliberate methodological constraint: absolute per-model performance is sacrificed in favor of a controlled comparison baseline. This criterion factors into the subsequent TTA stability analysis.

The complete wrapper integration code is publicly available at <https://github.com/XXXX/XXXX/>. Additionally, the modular wrappers can be downloaded directly as packages from PyPI and piwheels <https://www.piwheels.org/project/XXXX/> (and <https://www.piwheels.org/project/XXXX/>), while the reproducible experiment scripts and environment configurations are hosted at <https://github.com/XXXX/XXXX/>.

C. Evaluation & Metrics

The evaluation framework assesses three complementary dimensions: visual fidelity, computational efficiency, and memory consumption. Final metrics were computed exclusively on the held-out test set (section III-A), ensuring no overlap with the training views.

To quantify visual quality, this project used three standard novel view synthesis metrics: (i) PSNR (Peak Signal-to-Noise Ratio), to measure pixel-wise fidelity; (ii) SSIM (Structural Similarity Index Measure) [10], to evaluate structural coherence; and (iii) LPIPS (Learned Perceptual Image Patch Similarity) [11]. Following the protocol established by Niemeyer et al. [12], LPIPS was computed using Visual Geometry Group (VGG) features to accurately quantify perceptual distance.

Operational efficiency was measured through the inference rendering rate in frames per second (fps) across the test cameras, as did Kulhanek et al. [14]. For memory evaluation, both the static parameter footprint of the models and the dynamic VRAM allocation peak logged by PyTorch during runtime were analyzed.

To guarantee environmental isolation and reproducibility, all experiments were executed within Nvidia Docker containers, preventing external processes from adding noise to the performance metrics. The hardware setup consisted of an NVIDIA GeForce GTX 1660 Super (6GB VRAM) running CUDA 12.6.

IV. RESULTS AND DISCUSSION

This section presents the consolidated quantitative, computational and qualitative analyses across the evaluated LLFF scenes. Table IV summarizes the global averages for visual

fidelity, temporal efficiency, and memory footprint. The results reveal critical trade-offs between generalization, optimization stability, and hardware constraints.

A. Visual Fidelity

The per-scene architectures demonstrated vastly superior visual reconstruction capabilities. Nerfacto established the performance upper bound, achieving a global average of 27.11 dB PSNR and 0.875 SSIM (Table IV). In contrast, the generalizable models struggled significantly with the sparse constraints of the LLFF dataset.

PixelNeRF produced degenerate reconstructions across all LLFF scenes, consistent with its architectural dependency on dense multi-view overlap for accurate 2D-to-3D feature projection — a condition fundamentally incompatible with the $N \leq 10$ regime evaluated here. The purely local 2D feature projection via the frozen CNN encoder failed to triangulate depth accurately under extreme sparsity, resulting in severe isotropic blurring and a complete failure to reconstruct the underlying geometry. GNT, conversely, achieved structurally superior reconstructions relative to PixelNeRF within the same constrained optimization budget. Although not able to converge to the final scene’s minor details, the PSNR values ranged from 11.58 to 12.96, as observed in Table II. Interestingly, GNT’s TTA performance on the *room* scene degrades from 12.85 dB at $N = 6$ down to 11.58 dB at $N = 10$. We hypothesize that the fixed $\eta = 10^{-3}$, combined with a denser but still sparse view set, accelerates overfitting of the rendering layers, producing a local performance valley consistent with the representational collapse regime described in Section II. The inability to fully capture high-frequency, in-the-wild textures highlights the limitations of keeping the feature extraction network frozen during adaptation, forcing the model to rely on generalized priors that ultimately struggle to resolve out-of-distribution geometric complexities.

Furthermore, the stability of these visual results across different scenes is illustrated in Fig. 2. The boxplot highlights the significant performance variance inherent to each paradigm. Generalizable models (PixelNeRF and GNT) exhibit a highly constrained PSNR distribution, reflecting a consistent inability to capture high-frequency details regardless of the scene geometry. This variance reflects per-scene optimization sensitivity to geometric complexity; notably, even the worst-case per-scene PSNR exceeds the ceiling of all generalizable configurations. Instant-NGP achieved 19.33 dB in its worst scenario, whereas PixelNeRF’s absolute peak was bounded at 12.23 dB under any view budget.

B. Computational Trade-offs

From a systemic viability perspective, a clear dichotomy emerges between inference latency and training overhead. While generalizable models bypass extensive optimization (requiring only seconds for Zero-Shot inference or ~ 7 minutes for TTA), their volumetric feature aggregation mechanisms result in catastrophic inference bottlenecks, yielding rendering speeds below 0.02 FPS.

TABLE IV: Consolidated quantitative results across the LLFF dataset. Metrics represent the global average. Training time reflects the fine-tuning phase for TTA and full convergence for Per-Scene models. VRAM Peak denotes dynamic memory allocation during inference.

Paradigm	Model & Regime	PSNR (dB) $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Train / TTA Time	Inference (FPS) $\uparrow$	VRAM Peak (GB) $\downarrow$
Generalizable	PixelNeRF (Zero-Shot)	6.62	0.093	0.814	~ 22 s	0.004	1.93
	PixelNeRF (TTA)	6.38	0.071	0.801	4 min 23 s	0.006	1.97
	GNT (Zero-Shot)	10.73	0.300	0.760	~ 9 s	0.014	1.94
	GNT (TTA)	10.42	0.336	0.724	7 min 19 s	0.015	1.93
Per-Scene	Instant-NGP (100%)	14.06	0.442	0.498	31 min 51 s	0.199	1.09
	Nerfacto (100%)	27.11	0.875	0.171	8 min 57 s	0.388	1.43

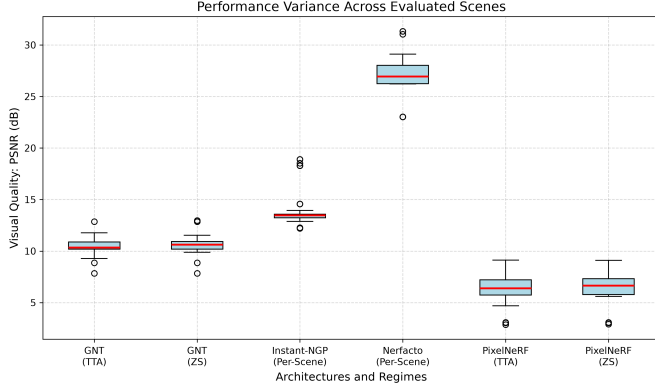


Fig. 2: Visual performance variance (PSNR) across all evaluated LLFF scenes.

Conversely, the explicit multiscale hash representations of the per-scene models proved vastly more efficient during run-time. Despite requiring lengthier convergence periods (from ~ 9 minutes for Nerfacto up to ~ 32 minutes for Instant-NGP), they achieve inference rates ranging from $13\times$ (Instant-NGP vs. GNT) to $97\times$ (Nerfacto vs. PixelNeRF) faster than the generalizable baselines. Furthermore, the hash-based models exhibit a highly optimized dynamic memory footprint. While Instant-NGP operates at just 1.10 GB of VRAM during inference on the *fern* scene (Table V), the global average footprint across all scenes scales slightly to 1.43 GB for Nerfacto (Table IV) due to structural complexity variance between captures. Nonetheless, these metrics solidify per-scene methods as the most viable path for hardware-constrained deployments despite the initial training cost.

C. Regime Trade-off Synthesis

The architectural trade-offs evaluated establish a rigid Pareto frontier between data availability and computational overhead. When multi-view data is abundant but inference latency and VRAM are strictly bounded, the *per-scene* paradigm (specifically Nerfacto) dominates the design space, delivering state-of-the-art fidelity with sub-second rendering. Conversely, under severe view scarcity ($N \leq 10$), the structural priors of *generalizable* models become indispensable to prevent total geometric collapse, yet they exact a severe penalty: inference speeds plummet to impractical levels (≤ 0.02 FPS) and visual

quality plateaus prematurely, as the frozen feature extractor cannot adapt its priors to scene-specific geometric complexities (a manifestation of the representational bottleneck described in Section II). Thus, the selection between regimes is not absolute, but entirely dictated by whether the deployment bottleneck lies in camera acquisition physics or rendering hardware capacity.

TABLE V: Computational Performance and Memory on the *fern* scene.

Regime	Model	Train Time (s) $\downarrow$	Inference (FPS) $\uparrow$	VRAM Peak (GB) $\downarrow$
Zero-Shot	GNT	8.77	0.007	1.97
TTA	GNT	392.15	0.008	1.97
Zero-Shot	PixelNeRF	39.02	0.003	1.60
TTA	PixelNeRF	496.82	0.006	1.92
Per-Scene	Instant-NGP	1707.91	0.183	1.09
Per-Scene	Nerfacto	423.45	0.371	1.10

D. Qualitative Analysis

The qualitative assessment visually corroborates the quantitative metrics, specifically illustrating the boundaries of each architectural paradigm. For visual clarity, we synthesized novel views matching the exact poses from the test set using perspective projection.

Fig. 3 highlights the severe disparities in reconstruction quality on the highly detailed *fern* scene. As visually evident, neither PixelNeRF nor GNT achieved proper convergence. PixelNeRF produced renderings with intense isotropic blurring, a typical artifact indicating the network’s failure to accurately localize 2D projected features across extremely sparse views. GNT, on the other hand, demonstrated a complete generalization failure regarding the *in-the-wild* characteristics of the scene. This resulted in a representational visual collapse, heavily distorting the volumetric geometry into noise—which persisted even after the 10-view Test-Time Adaptation (TTA).

Conversely, the per-scene architectures successfully captured the intricate foliage geometry. Nerfacto achieved the highest perceptual fidelity across all scenes (PSNR 27.11 dB, SSIM 0.875, LPIPS 0.171), recovering fine-grained foliage details consistent with the ground truth resolution (Fig. 3, bottom right).

V. CONCLUSION

This work developed a methodology for systematically characterizing the trade-offs between computational performance

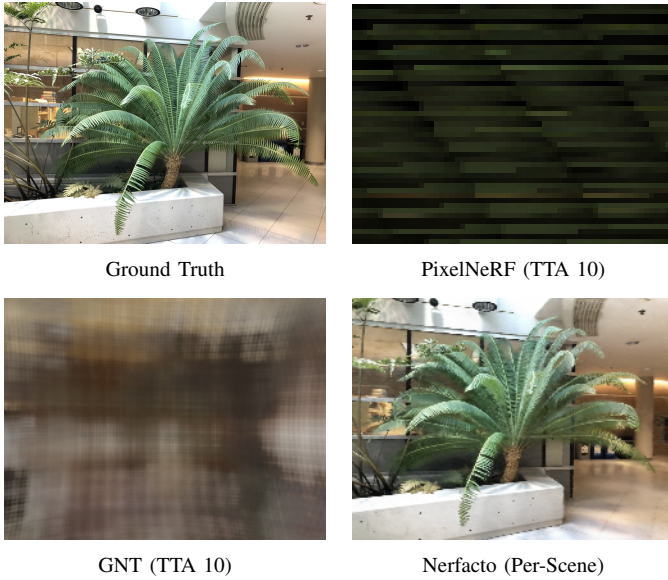


Fig. 3: Qualitative comparison on the *fern* scene. Generalizable architectures (PixelNeRF and GNT) failed to converge under sparse constraints, resulting in severe blurring and visual collapse. In contrast, per-scene optimization models (specifically Nerfacto) successfully reconstructed the complex geometry.

and reconstruction fidelity of generalizable and Per-Scene NeRF architectures under sparse-view conditions. The experimental results demonstrate that Per-Scene models achieve substantially lower inference latency and VRAM consumption than generalizable models, making them well suited for deployment in resource-constrained environments. However, these benefits rely on dense multi-view supervision, restricting their use in sparse-view acquisition scenarios, where generalizable architectures remain a practical alternative despite reduced reconstruction fidelity.

In addition, although the proposed strategy provides a standardized and reproducible framework, some limitations should be acknowledged. The restricted hardware configuration of the experimental environment (6 GB VRAM) prevented the inclusion of architectures that rely on computationally intensive 3D cost volumes or strong geometric regularization, such as RegNeRF. Furthermore, the experimental validation was conducted on the forward-facing LLFF dataset, and the adopted sparse-view sampling strategy exploits the sequential ordering of its image indices, which may limit the direct applicability of the proposed methodology to datasets composed of unordered image collections.

In this context, future work will focus on extending the proposed methodology to unbounded 360-degree datasets and more diverse acquisition scenarios, enabling a broader characterization of the relationship between computational efficiency and reconstruction fidelity across neural rendering paradigms. Also, diffusion priors will be investigated as regularization mechanisms during Test-Time Adaptation to improve reconstruction quality under extremely sparse-view conditions.

ACKNOWLEDGMENT

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- [2] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. Mcallister, J. Kerr, and A. Kanazawa, “Nerfstudio: A modular framework for neural radiance field development,” in *Special Interest Group on Computer Graphics and Interactive Techniques Conference Proceedings*, ser. SIGGRAPH ’23. ACM, Jul. 2023, p. 1–12. [Online]. Available: <http://dx.doi.org/10.1145/3588432.3591516>
- [3] Adobe, *Substance 3D Viewer (beta)*, 2025, computer Software. [Online]. Available: <https://helpx.adobe.com/substance-3d-viewer/get-started/quick-start.html>
- [4] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1–15, Jul. 2022. [Online]. Available: <http://dx.doi.org/10.1145/3528223.3530127>
- [5] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, “pixelnerf: Neural radiance fields from one or few images,” 2021. [Online]. Available: <https://arxiv.org/abs/2012.02190>
- [6] Z. Wang, M. Varma, P. Wang, X. Chen, T. Chen, and S. Venugopalan, “Is attention all that nerf needs?” 2023. [Online]. Available: <https://arxiv.org/abs/2207.13298>
- [7] K. Schwarz, Y. Liao, M. Niemeyer, and A. Geiger, “Graf: Generative radiance fields for 3d-aware image synthesis,” 2021. [Online]. Available: <https://arxiv.org/abs/2007.02442>
- [8] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan, and L. Zhang, “Cvt: Introducing convolutions to vision transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 22–31.
- [9] P. Hedman, P. P. Srinivasan, B. Mildenhall, J. T. Barron, and P. Debevec, “Baking neural radiance fields for real-time view synthesis,” 2021. [Online]. Available: <https://arxiv.org/abs/2103.14645>
- [10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [11] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.
- [12] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. Sajjadi, A. Geiger, and N. Radwan, “Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5480–5490.
- [13] H. Heo, T. Kim, J. Lee, J. Lee, S. Kim, H. Kim, and J.-H. Kim, “Robust camera pose refinement for multi-resolution hash encoding,” 02 2023.
- [14] J. Kulhanek and T. Sattler, “Nerfbaselines: Consistent and reproducible evaluation of novel view synthesis methods,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.17345>
- [15] A. Laurino, A. Franceschini, L. Pesce, L. Cinci, A. Montalbano, G. Mazzamuto, G. Sancataldo, G. Nesi, I. Costantini, L. Silvestri, and F. S. Pavone, “A guide to perform 3d histology of biological tissues with fluorescence microscopy,” *International Journal of Molecular Sciences*, vol. 24, no. 7, 2023. [Online]. Available: <https://www.mdpi.com/1422-0067/24/7/6747>
- [16] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” 2020. [Online]. Available: <https://arxiv.org/abs/2003.08934>
- [17] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, “Local light field fusion: Practical view synthesis with prescriptive sampling guidelines,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.00889>
- [18] T. Müller, “tiny-cuda-nn,” <https://github.com/NVlabs/tiny-cuda-nn>, 2021, version 1.7, BSD-3-Clause License.